

Java vs Go

A Java developer's quick reference | Mathieu Neron | 2026

This cheatsheet anchors Go on constructs you already know in Java. Left column: Java. Right column: Go (1.22+). Go is intentionally minimal — the biggest mental shifts for a Java dev are: **no inheritance**, **errors as return values**, **structural interfaces**, **goroutines** + **channels** for concurrency.

1. Hello world & program entry

Program entry point

Java

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, " + args[0]);
    }
}
// javac Hello.java && java Hello world
```

Go

```
package main

import (
    "fmt"
    "os"
)

func main() {
    fmt.Println("Hello,", os.Args[1])
}
// go run hello.go world / go build && ./hello world
```

File / module / package layout

Java

```
// One public class per file.
// src/main/java/com/acme/Hello.java
package com.acme;
// pom.xml or build.gradle for deps
```

Go

```
// Each directory == one package. All .go files in
// the dir share the package namespace.
// go.mod at module root:
module github.com/acme/svc
go 1.22
require github.com/foo/bar v1.2.3
```

2. Variables, constants, type inference

Local variables

Java

```
int x = 5;
final String s = "hi";
var list = new ArrayList<Integer>(); // Java 10+
```

Go

```
var x int = 5
var s = "hi" // type inferred
y := 5 // short decl, only inside funcs
const Pi = 3.14159 // typed by use site
const (
    A = iota // 0
    B // 1
    C // 2
)
```

Zero values & visibility

Java

```
// Fields default: 0, false, null. Locals must be init'd.
// Visibility via keywords: public/protected/private.
public class C { private int x; }
```

Go

```
// All declared vars get the type's zero value
// (0, "", false, nil). No null-pointer surprises on init.
// Visibility: Capitalized = exported, lowercase = unexported.
type C struct { x int } // x unexported (private)
type D struct { X int } // X exported (public)
```

3. Primitive types & conversions

Numeric types

Java

```
byte b; short s; int i; long l;
float f; double d;
int n = Integer.parseInt("42");
double x = Double.parseDouble("1.5");
// Implicit widening: int -> long. Narrowing requires cast.
long L = i;    int j = (int) L;
```

Go

```
// int8/16/32/64, uint*, float32/64, complex64/128
// `int`/`uint` are platform-width (usually 64).
import "strconv"
n, err := strconv.Atoi("42")
x, err := strconv.ParseFloat("1.5", 64)
// NO implicit conversions, even widening:
var i int32 = 5
var L int64 = int64(i)    // explicit
```

Bool, byte, rune, string

Java

```
boolean b = true;
char c = 'A';           // 16-bit UTF-16 unit
byte[] bs = "hi".getBytes(StandardCharsets.UTF_8);
String s = new String(bs, StandardCharsets.UTF_8);
```

Go

```
b := true
c := 'A'           // rune == int32 (Unicode pt)
bs := []byte("hi") // UTF-8 bytes
s := string(bs)
// Strings are immutable byte sequences (typically UTF-8).
// for i, r := range s iterates runes, not bytes.
```

4. Strings

Interpolation & formatting

Java

```
String name = "world";
String s1 = "Hello, " + name;
String s2 = String.format("%s = %.2f", "pi", 3.14);
// Java 15+ text block:
String body = """
    Hello,
    World
    """;
```

Go

```
name := "world"
s1 := "Hello, " + name
s2 := fmt.Sprintf("%s = %.2f", "pi", 3.14)
// Raw string literal (backticks): keeps newlines, no escapes
body := `
    Hello,
    World
`
```

Common operations

Java

```
s.length(); s.toUpperCase(); s.trim();
s.startsWith("H"); s.contains("ll");
String[] parts = s.split(",");
String joined = String.join("-", parts);
```

Go

```
import "strings"
len(s)           // BYTES (not runes)
strings.ToUpper(s); strings.TrimSpace(s)
strings.HasPrefix(s, "H"); strings.Contains(s, "ll")
parts := strings.Split(s, ",")
joined := strings.Join(parts, "-")
```

5. Control flow

If / else (no ternary)

Java

```
if (x > 0) { ... }
else if (x == 0) { ... }
else { ... }

int sign = x > 0 ? 1 : (x < 0 ? -1 : 0);
```

Go

```
if x > 0 {
    ...
} else if x == 0 {
    ...
} else { ... }

// No ternary. Idiom: a tiny helper or explicit if.
sign := 0
if x > 0 { sign = 1 } else if x < 0 { sign = -1 }

// "if with init" is a Go idiom:
if v, err := f(); err == nil { use(v) }
```

Switch

Java

```
String label = switch (x) {
    case 1, 2 -> "small";
    case 3    -> "three";
    default   -> "other";
};
// Pattern switch (21+):
switch (s) {
    case Circle c -> ...;
    case null     -> ...;
}
```

Go

```
switch x {
case 1, 2:
    label = "small"
case 3:
    label = "three"
default:
    label = "other"
}

// No implicit fallthrough; use `fallthrough` keyword.
// "Type switch":
switch v := s.(type) {
case Circle: ...
case nil:    ...
}
```

6. Loops (just ‘for’)

All loops use ‘for’

Java

```
for (int i = 0; i < 10; i++) { ... }
while (cond) { ... }
do { ... } while (cond);
for (var x : xs) { ... }
```

Go

```
for i := 0; i < 10; i++ { ... } // classic
for cond { ... }                // == while
for { ... }                     // == while(true)
for i, x := range xs { ... }    // index, value
for k, v := range m { ... }     // map
for _, x := range xs { ... }    // discard index
// 1.22+: each loop iter creates fresh `i` (no aliasing trap)
```

7. Collections & data structures

Arrays vs slices vs maps

Java

```
int[] a = {1, 2, 3}; // fixed
List<Integer> xs = new ArrayList<>(List.of(1,2,3));
xs.add(4); xs.get(0); xs.size();
Set<String> set = new HashSet<>(List.of("a","b"));
Map<String,Integer> m = new HashMap<>();
m.put("a", 1);
```

Go

```
a := [3]int{1, 2, 3} // ARRAY (fixed size)
xs := []int{1, 2, 3} // SLICE (dynamic view)
xs = append(xs, 4); xs[0]; len(xs); cap(xs)
xs[1:3] // sub-slice (no copy!)

// No built-in set; use map[T]struct{}
set := map[string]struct{}{"a": {}, "b": {}}
_, ok := set["a"] // membership

m := map[string]int{"a": 1}
m["a"] = 1; v, ok := m["a"]; delete(m, "a")
```

Slice gotcha

Java

```
// subList returns a view; modifying it mutates the parent.  
// Same general "view vs copy" hazard, less surprising  
// because typical code uses fresh ArrayList copies.
```

Go

```
// Slices share backing array. Sub-slicing is a VIEW.  
a := []int{1,2,3,4,5}  
b := a[1:3] // {2,3}, shares storage  
b[0] = 99 // mutates a[1] too!  
// To fully detach: copy  
c := make([]int, len(b)); copy(c, b)
```

8. Functions

Definition, varargs, multiple returns

Java

```
int add(int a, int b) { return a + b; }  
int sum(int... xs) {  
    int t = 0; for (var x : xs) t += x; return t;  
}  
sum(1, 2, 3);  
// No multi-return: use record or out-params.
```

Go

```
func add(a, b int) int { return a + b }  
  
func sum(xs ...int) int {  
    t := 0  
    for _, x := range xs { t += x }  
    return t  
}  
  
// Multi-return is idiomatic, especially (T, error):  
func divmod(x, y int) (int, int) { return x/y, x%y }  
q, r := divmod(10, 3)
```

Lambdas / closures

Java

```
Function<Integer,Integer> sq = x -> x * x;  
list.forEach(System.out::println);
```

Go

```
sq := func(x int) int { return x * x }  
adder := func() func(int) int {  
    sum := 0  
    return func(x int) int { sum += x; return sum }  
}()  
adder(2); adder(3) // 2, 5 (closure captures sum)
```

9. Structs & interfaces (NO inheritance)

Class vs struct + methods

Java

```
public class Point {  
    private final double x, y;  
    public Point(double x, double y) {  
        this.x = x; this.y = y;  
    }  
    public double dist(Point o) {  
        return Math.hypot(x - o.x, y - o.y);  
    }  
}
```

Go

```
type Point struct {  
    X, Y float64 // exported fields  
}  
  
// Method = func with a receiver  
func (p Point) Dist(o Point) float64 {  
    dx, dy := p.X-o.X, p.Y-o.Y  
    return math.Sqrt(dx*dx + dy*dy)  
}  
  
// Constructor: just a function, no `new` keyword needed  
func NewPoint(x, y float64) Point { return Point{X: x, Y: y} }
```

Pointer vs value receiver

Java

```
// Java: objects always passed by reference (the reference  
// is by value). No equivalent distinction.
```

Go

```
type Counter struct { n int }  
func (c Counter) GetVal() int { return c.n } // value rcv  
func (c *Counter) Inc() { c.n++ } // ptr rcv  
// Use pointer receivers when method mutates state OR  
// the struct is large. Be consistent within a type.
```

Interfaces (structural, implicit)

Java

```
public interface Stringer { String toString(); }
public class Foo implements Stringer {
    public String toString() { return "foo"; }
}
// Nominal: must declare "implements".
```

Go

```
type Stringer interface {
    String() string
}

type Foo struct{}
func (f Foo) String() string { return "foo" }
// Foo SATISFIES Stringer automatically. No "implements".
// Interfaces are STRUCTURAL: shape, not name, matches.
```

Composition replaces inheritance

Java

```
class Animal { void breathe() { ... } }
class Dog extends Animal { void bark() { ... } }
Dog d = new Dog(); d.breathe(); d.bark();
```

Go

```
type Animal struct{}
func (a Animal) Breathe() { ... }

type Dog struct {
    Animal // embedded type (anonymous field)
}
func (d Dog) Bark() { ... }

d := Dog{}
d.Breathe() // promoted from embedded Animal
d.Bark()
// No virtual dispatch. Method set "promoted" via embedding.
```

10. Generics (Go 1.18+)

Generic function & type

Java

```
<T extends Comparable<T>> T max(T a, T b) {
    return a.compareTo(b) >= 0 ? a : b;
}
class Box<T> { final T v; Box(T v) { this.v = v; } }
```

Go

```
func Max[T cmp.Ordered](a, b T) T {
    if a >= b { return a }
    return b
}

type Box[T any] struct { V T }
b := Box[int]{V: 5}
```

Constraints

Java

```
// Constraint = upper bound on a class hierarchy.
<T extends Number & Comparable<T>>
T sum(List<T> xs) { ... }
```

Go

```
// Constraints are interfaces (possibly with type sets):
type Number interface {
    ~int | ~int64 | ~float32 | ~float64
}
func Sum[T Number](xs []T) T {
    var t T
    for _, x := range xs { t += x }
    return t
}
```

11. Error handling (no exceptions)

Errors are values

Java

```
try (var f = Files.newBufferedReader(p)) {
    return f.readLine();
} catch (IOException e) {
    log.error("read failed", e);
    throw new MyException("oops", e);
}
```

Go

```
data, err := os.ReadFile(p)
if err != nil {
    return fmt.Errorf("read failed: %w", err) // wrap
}
// No try/catch. Functions return (T, error).
// Caller MUST check err. Idiom: "if err != nil { return }"
```

Custom error & sentinel checks

Java

```
public class NotFound extends RuntimeException { ... }
catch (NotFound e) { ... }
catch (RuntimeException e) { ... }
```

Go

```
var ErrNotFound = errors.New("not found")
type ValidationError struct{ Field string }
func (e *ValidationError) Error() string { ... }

if errors.Is(err, ErrNotFound) { ... }
var ve *ValidationError
if errors.As(err, &ve) { use(ve.Field) }
```

Cleanup: defer (replaces try-with-resources)

Java

```
try (var f = new FileInputStream(p)) {
    // f auto-closed on exit
}
```

Go

```
f, err := os.Open(p)
if err != nil { return err }
defer f.Close() // runs when func returns
// Multiple defers run in LIFO order. Args evaluated NOW.
```

Panic / recover (rare)

Java

```
// throw new RuntimeException(...); is the analog,
// but Java idiom uses exceptions widely.
```

Go

```
panic("unrecoverable") // nuke goroutine
defer func() {
    if r := recover(); r != nil { ... }
}()
// Reserve panic for truly unrecoverable bugs.
// Normal errors -> return error.
```

12. Modules / packages / imports

Imports

Java

```
package com.acme.svc;
import java.util.List;
import java.util.stream.Collectors;
// Maven coords in pom.xml. Repackaged into uberjar.
```

Go

```
package svc

import (
    "fmt"
    "time"
    "github.com/acme/lib" // module path
    log "github.com/sirupsen/logrus" // alias
)
// go mod tidy resolves go.mod / go.sum
```

13. Concurrency (goroutines + channels)

Goroutines vs threads

Java

```
Thread t = new Thread() -> work();
t.start(); t.join();

ExecutorService pool = Executors.newFixedThreadPool(4);
Future<Integer> f = pool.submit() -> compute();
int v = f.get();
```

Go

```
go work() // fire-and-forget goroutine
// Goroutines are cheap (~few KB stack). Spawn millions.
// No "join" - coordinate via channels or sync.WaitGroup.

var wg sync.WaitGroup
for _, item := range items {
    wg.Add(1)
    go func(x Item) { defer wg.Done(); process(x) }(item)
}
wg.Wait()
```

Channels

Java

```
BlockingQueue<Integer> q = new ArrayBlockingQueue<>(8);
q.put(42);
int v = q.take();
```

Go

```
ch := make(chan int, 8) // buffered (8) channel
ch <- 42                // send
v := <-ch               // receive
close(ch)              // close (only sender)
for v := range ch { ... } // drain until closed

// "Don't communicate by sharing memory; share memory by
// communicating." - channels are the primary primitive.
```

Select (multiplex)

Java

```
// Closest analog: CompletableFuture.anyOf(...).
CompletableFuture.anyOf(f1, f2)
    .thenAccept(System.out::println);
```

Go

```
select {
case v := <-ch1:
    use(v)
case v := <-ch2:
    use(v)
case <-time.After(2 * time.Second):
    return errors.New("timeout")
case <-ctx.Done():
    return ctx.Err()
}
```

Context (cancel + deadline)

Java

```
// Roughly: Future.cancel(true), interrupt flag, or
// scheduled-future timeouts. No standardized propagation.
```

Go

```
ctx, cancel := context.WithTimeout(parent, 5*time.Second)
defer cancel()
result, err := callRPC(ctx, req)
// Context flows through the call tree as the first arg.
// Idiom: func F(ctx context.Context, ...) (...).
```

Mutex / atomic

Java

```
synchronized (lock) { ... }
ReentrantLock lk = new ReentrantLock();
AtomicInteger ai = new AtomicInteger();
ai.incrementAndGet();
```

Go

```
var mu sync.Mutex
mu.Lock(); defer mu.Unlock()
// Run with `go test -race` to catch data races.

import "sync/atomic"
var n atomic.Int64
n.Add(1); v := n.Load()
```

14. Idiom appendix

nil / equality / common gotchas

- **nil is typed.** A var p *T is nil; an interface value is nil only if both its type AND value are nil. Returning (*MyErr)(nil) as error is NOT nil — common bug.
- **Equality.** == compares structs field-by-field, but only if all fields are comparable (no slices/maps/funcs). Slices/maps/funcs are not ==-comparable; use reflect.DeepEqual or hand-roll.
- **Range copies.** for _, x := range xs gives x as a copy. Take &xs[i] or use index for mutation. (Pre-1.22 also: same loop variable address reused; fixed in 1.22.)
- **No constructors.** Just write a NewT(...) function returning T or *T.
- **Errors are values, not control flow.** Wrap with fmt.Errorf("ctx: %w", err); unwrap with errors.Is/As. Never use panic for normal flow.
- **Goroutine leaks.** A goroutine that blocks on a never-closed channel never exits. Always have a cancellation path (ctx or close-the-channel).
- **Capitalization == visibility.** Foo exported, foo package-private. JSON tags use struct tags: `json:"foo,omitempty"`.
- **Tooling.** go fmt (or gofmt) is non-negotiable; go vet, staticcheck, go test, go test -race, go build, go mod tidy.